

Selezione dinamica del batch size in metodi di ottimizzazione

Corso di laurea in Scienze Matematiche per l'Intelligenza Artificiale, A.A. 2025/2026

Alessandro Lo Curcio

Relatore: Prof. Marco Sciandrone

Contesto

Obiettivo: minimizzare la funzione di loss $J(w) = \frac{1}{N} \sum_{i=1}^N \ell(w; i)$, $w \in \mathbb{R}^m$

Con $N \approx 10^6 - 10^9$ esempi ed m grande: il gradiente completo di J costa $O(Nm)$. per iterazione, proibitivo

SGD (Robbins–Monro, 1951) $w_{k+1} = w_k - \alpha_k \frac{1}{N} \sum_{i=1}^N \nabla \ell(w_k; i)$

Mini-batch S_k di dimensione n_k : stima $g_k = \nabla J_{S_k}(w_k) = \frac{1}{n_k} \sum_{i \in S_k} \nabla \ell(w_k; i)$ costo $O(n_k m)$. Se $n_k \ll N$ le iterazioni sono economiche.

Compromesso classico (Bottou–Bousquet, 2008): batch piccolo = stima rumorosa; batch grande = stima accurata ma costo proporzionale alla dimensione.

Nei metodi classici n_k è fissata a priori; questa tesi analizza la proposta di renderla dinamica durante le iterazioni (Byrd, Chin, Nocedal e Wu, 2012).

Metodi a varianza ridotta

SVRG (Stochastic Variance Reduced Gradient; Johnson–Zhang, 2013) e SAGA (metodo di riduzione della varianza di Defazio–Bach–Lacoste-Julien, 2014): SGD + correzione costruita su quantità globali.

$$g_k = \nabla \ell(w_k; i_k) - c_k + \mathbb{E}[c_k], \quad \mathbb{E}[g_k] = \nabla J(w_k)$$

Se la correzione c_k è correlata al rumore del gradiente stocastico, la varianza di g_k si riduce: si parla di variance reduction

$$\text{SVRG: } g_k = \nabla \ell(w_k; i_k) - \nabla \ell(\tilde{w}; i_k) + \nabla J(\tilde{w})$$

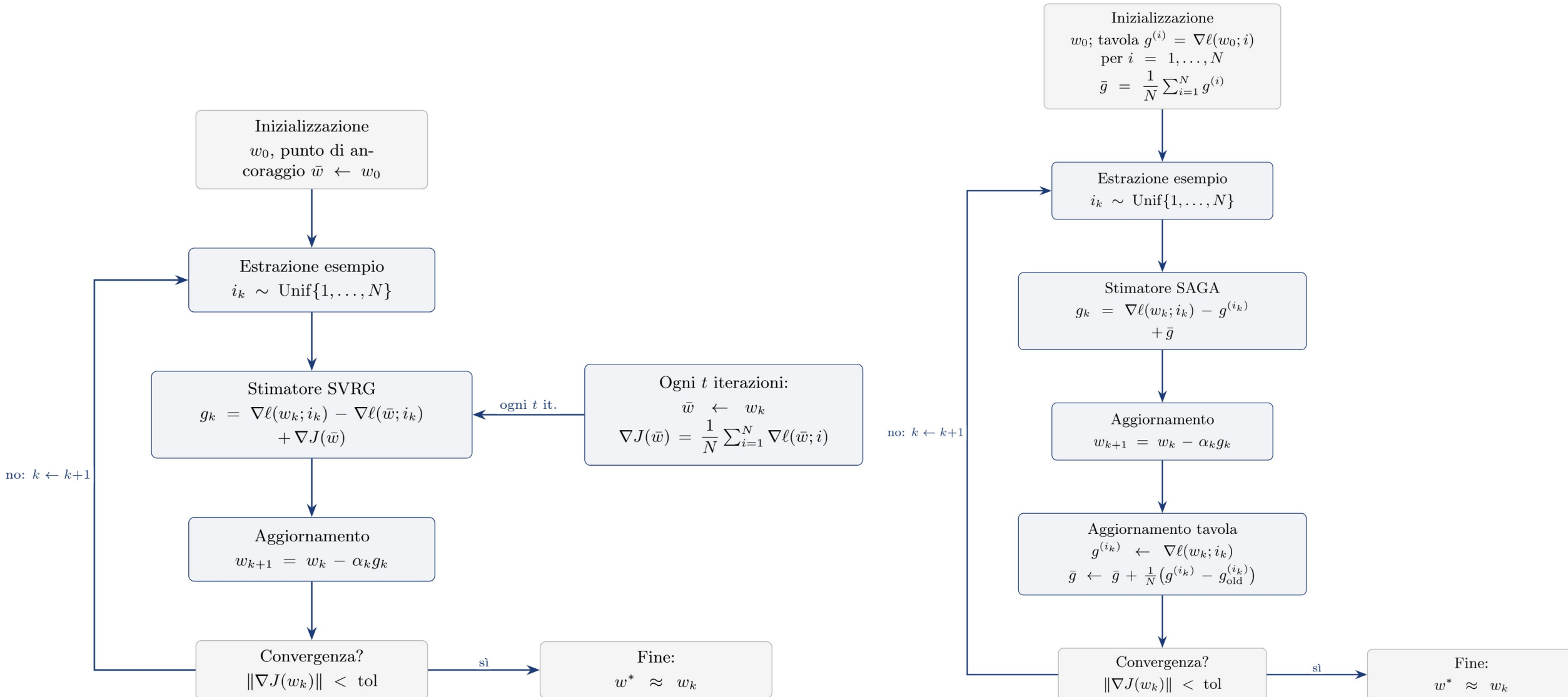
Finché $w_k \approx \tilde{w}$, per la regolarità dei gradienti, $\nabla \ell(w_k; i_k) \approx \nabla \ell(\tilde{w}; i_k)$. I due termini stocastici si elidono e resta $g_k \approx \nabla J(\tilde{w})$: poca varianza.

$$\text{SAGA (Stochastic Average Gradient): } g_k = \nabla \ell(w_k; i_k) - g^{(i_k)} + \bar{g}$$

SVRG: $O(m)$ memoria, $O(Nm)$ ogni t iterazioni.

SAGA: $O(Nm)$ memoria, nessuna passata periodica.

Metodi a varianza ridotta: Schemi



Condizione di Controllo sulla Varianza

Se $\|e_k\|_2 \leq \theta \|g_k\|_2$ (con $\theta \in (0,1)$, $e_k = g_k - \nabla J(w_k)$) allora $-g_k$ è di discesa per J

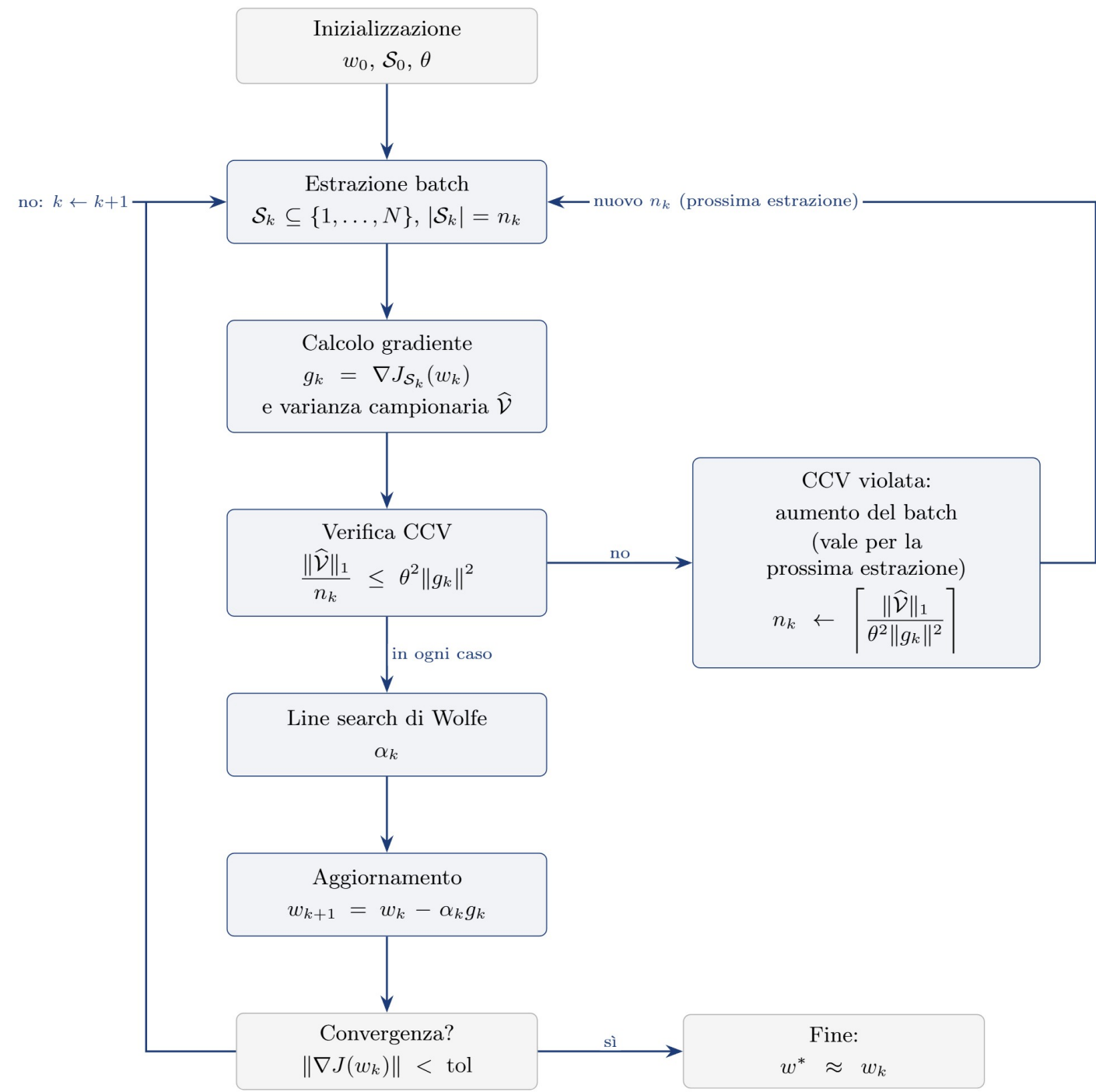
Non potendo calcolare e_k esattamente, calcoliamo $\mathbb{E}[\|e_k\|_2^2] = \text{Var}(e_k) = \text{Var}(g_k)$

$= \text{Var}\left(\frac{1}{n_k} \sum_{i \in S_k} \nabla \ell(w_k; i)\right)$, ora n_k viene portata fuori (al quadrato) e $\sum_{i \in S_k} \text{Var}(\nabla \ell(w_k; i)) \approx n_k \cdot \|\hat{v}\|_1$

Dove $\hat{v} := \frac{1}{n_k-1} \sum_{i \in S_k} (\nabla \ell(w_k; i) - g_k)^2$

Quindi la condizione diventa $\frac{\|\hat{v}\|_1}{n_k} \leq \theta^2 \|g_k\|_2^2$, se non è soddisfatta $n_k^{\text{new}} = \left\lceil \frac{\|\hat{v}\|_1}{\theta^2 \|g_k\|_2^2} \right\rceil$

Dynamic GD



Analisi Deterministica

Il fattore di contrazione è più piccolo di quello di Nocedal. Di conseguenza, il nostro bound sull'errore decade più velocemente a ogni iterazione, a parità di passo e di ipotesi sulla funzione.

Ottenuto usando Polyak–Lojasiewicz $\|\nabla J(w)\|_2^2 \geq 2\lambda J(w)$

anziché la versione più debole usata da Nocedal e Wright. Questa disuguaglianza vale sotto le stesse ipotesi del paper originale ovvero $\exists \lambda, L \in \mathbb{R}, 0 < \lambda \leq L$, tale che $\forall w, d \in \mathbb{R}^m$

$$\lambda \|d\|_2^2 \leq d^T \nabla^2 J(w) d \leq L \|d\|_2^2.$$

Numero di iterazioni dimezzato

Teorema

Teorema di Convergenza Deterministica. Sia $J : \mathbb{R}^m \rightarrow \mathbb{R}$ una funzione λ -fortemente convessa ($\nabla^2 J(w) \succeq \lambda I$ per ogni w) e con gradiente L -Lipschitz ($\nabla^2 J(w) \preceq LI$ per ogni w). Si assuma che a ogni iterazione valga la condizione di accuratezza e si scelga il passo $\alpha = \frac{1-\theta}{L}$. Allora

$$\underbrace{J(w_{k+1}) - J(w^*)}_{=:\varepsilon_{k+1}} \leq \left(1 - \frac{(1-\theta)\lambda}{L}\right) \underbrace{(J(w_k) - J(w^*))}_{=:\varepsilon_k}.$$

Di conseguenza:

- Il fattore di riduzione per iterazione è $\rho = 1 - \frac{(1-\theta)\lambda}{L} \in (0, 1)$;
- Il numero di iterazioni per ottenere $J(w_k) - J(w^*) \leq \varepsilon$ è al più

$$k \geq \frac{L}{(1-\theta)\lambda} \log\left(\frac{J(w_0) - J(w^*)}{\varepsilon}\right) = O\left(\frac{L}{\lambda} \log \frac{1}{\varepsilon}\right).$$

Analisi Stocastica

Ipotesi strutturali su $J(w)$: $\lambda I \leq \nabla^2 J(w) \leq LI$, $\forall w \in \mathbb{R}^m$, $0 < \lambda \leq L$.

Ipotesi stocastiche: $\exists \omega > 0$ tale che $\|\text{Var}(\nabla \ell(w_k; I))\|_1 \leq \omega$, $\forall w_k$.

$$\exists a \in \mathbb{R}, a > 1, \text{ t.c. } n_k = \lceil a^k \rceil, \quad \forall k \geq 0$$

Ricorrenza $\varepsilon_{k+1} \leq \left(1 - \frac{\lambda}{L}\right) \varepsilon_k + \frac{\omega}{2Ln_k}$, $\varepsilon_k := \mathbb{E}[J(w_k) - J(w_*)]$,
:

Migliorata con PL (il coefficiente di ε_k è più piccolo: $\alpha = 1 - \frac{\lambda}{L}$ invece di $1 - \frac{\lambda}{2L}$)

$$\varepsilon_k \leq C\rho^k, \quad \rho = \max\left\{1 - \frac{\lambda}{L}, \frac{1}{a}\right\}, \text{ invece di } \rho_N = \max\left\{1 - \frac{\lambda}{4L}, \frac{1}{a}\right\}.$$

Migliorata risolvendo la ricorrenza (soluzione in forma chiusa, senza induzione)

Newton-CG

Risolvi il sistema $J_{\mathcal{H}_k}(w_k) d_k = -\nabla J_{\mathcal{S}_k}(w_k)$ con il metodo del gradiente coniugato Hessian free

$$\nabla^2 J_{\mathcal{S}_k}(w_k) d + \nabla J_{\mathcal{S}_k}(w_k) = \underbrace{\nabla^2 J_{\mathcal{H}_k}(w_k) d + \nabla J_{\mathcal{S}_k}(w_k)}_{\text{Residuo del CG}} + \underbrace{(\nabla^2 J_{\mathcal{S}_l})}_{\text{Hessiana di } \mathcal{S}_l}$$

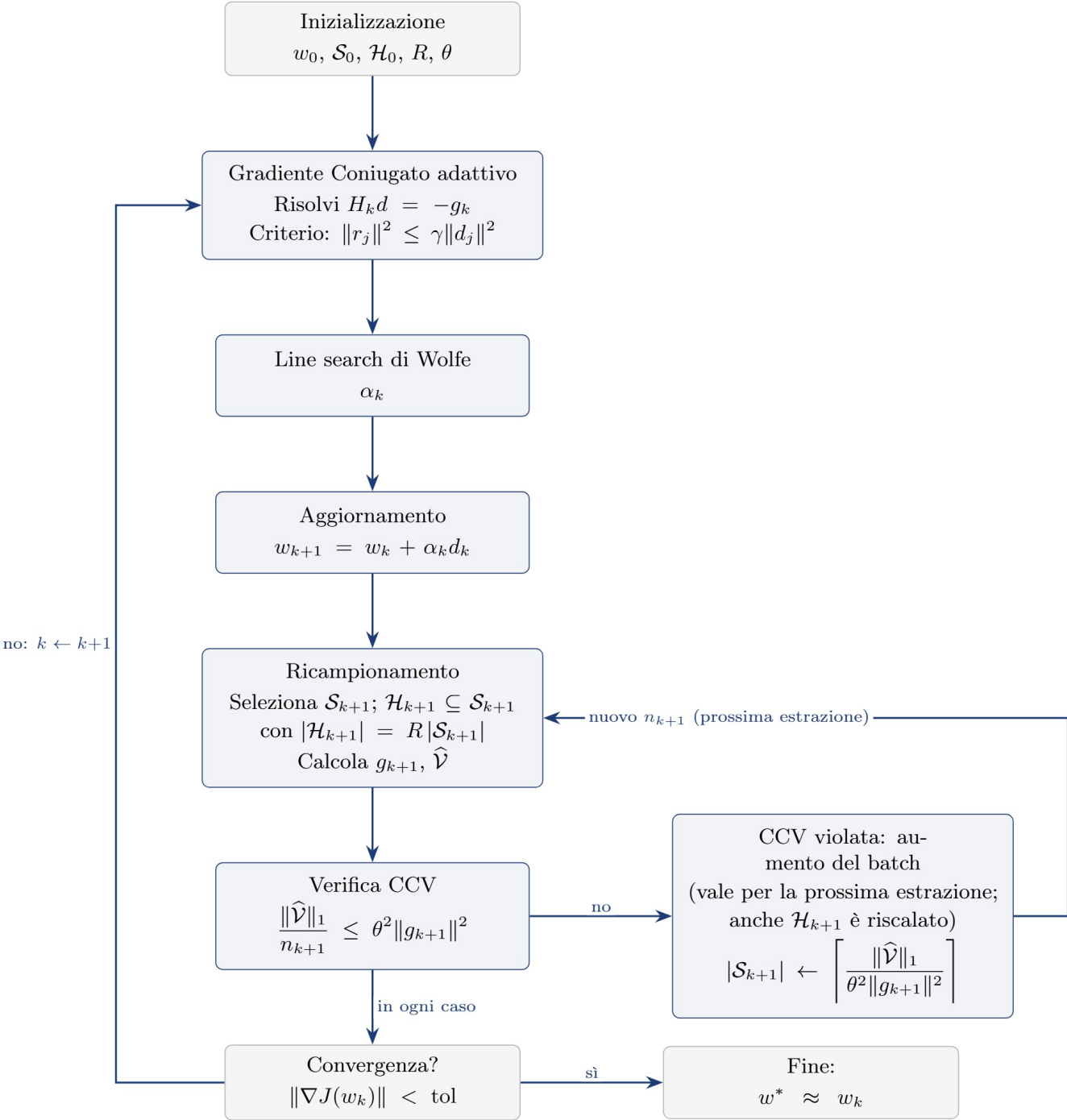
Il metodo CG si ferma quando l'errore è confrontabile con $\mathbb{E} \left[\|\Delta_{\mathcal{H}_k}(w_k; d)\|_2^2 \right] \approx \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d)\|_1}{|\mathcal{H}_k|}$

Nella prima iterazione il residuo è $g_k = \nabla J_{\mathcal{S}_k}(w_k)$ (partiamo da $d_0=0$), sfruttando l'omogeneità della varianza, possiamo calcolarla sulla prima direzione di ricerca $p_0 = -g_k$,

e per le direzioni successive d_j usiamo l'approssimazione $\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d_j)\|_1 \approx \alpha \|d_j\|_2^2$

con $\alpha := \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{\|p_0\|_2^2}$, poniamo $\gamma := \frac{\alpha}{|\mathcal{H}_k|}$

Newton-CG: Schema



Newton-CG L1

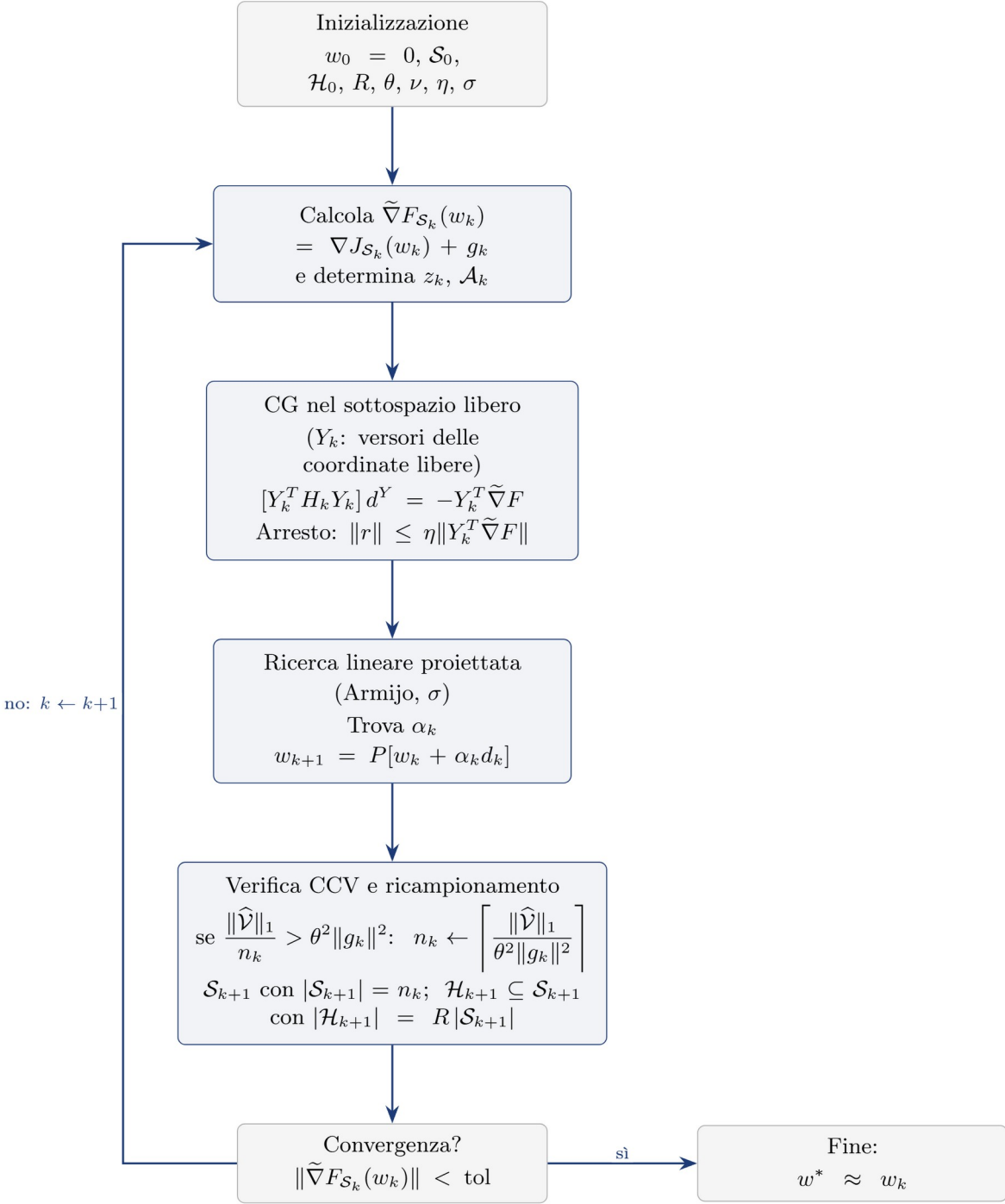
$$F(w) = J(w) + \nu \sum_{i=1}^m |w_i|$$

$$[\bar{\nabla} F(w)]_i = \underbrace{\frac{\partial J(w)}{\partial w_i}}_{\text{gradiente di } J} + \underbrace{g_i}_{\text{subgradiente di } \nu|w_i|}, \quad g_i \in \begin{cases} \{+\nu\}, & w_i > 0, \\ \{-\nu\}, & w_i < 0, \\ [-\nu, \nu] & w_i = 0. \end{cases}$$

Quando $w_i = 0$ c'è libertà su $g_i \in [-\nu, \nu]$; la scelta è fatta in modo che in w^* valga $\bar{\nabla} F(w^*) = 0$. Posto $d_i := \frac{\partial J(w)}{\partial w_i}$ (così $[\bar{\nabla} F]_i = d_i + g_i$), si prende $g_i = \arg \min_{g \in [-\nu, \nu]} |d_i + g|$: il subgradiente che rende $d_i + g_i$ il più vicino possibile a zero, così $w_i = 0$ è riconosciuto come vero minimo quando lo è. Definiamo ora il vettore che codifica la decisione che prendiamo su ogni variabile (tenerla libera o bloccarla)

$$z_k^i = \begin{cases} +1 & w_k^i > 0 \text{ oppure } \left(w_k^i = 0 \text{ e } \frac{\partial J_{\mathcal{S}_k}}{\partial w_i} < -\nu \right) \\ -1 & w_k^i < 0 \text{ oppure } \left(w_k^i = 0 \text{ e } \frac{\partial J_{\mathcal{S}_k}}{\partial w_i} > \nu \right) \\ 0 & \text{altrimenti} \end{cases}$$

Newton-CG L1: Schema



Condizione su d_i	g_i scelto	z_k^i
$d_i < -\nu$	$g_i = +\nu$	+1 (libera a destra)
$d_i > +\nu$	$g_i = -\nu$	-1 (libera a sinistra)
$-\nu \leq d_i \leq \nu$	$g_i = -d_i$	0 (blocca a zero)

Barzilai-Borwein CCV

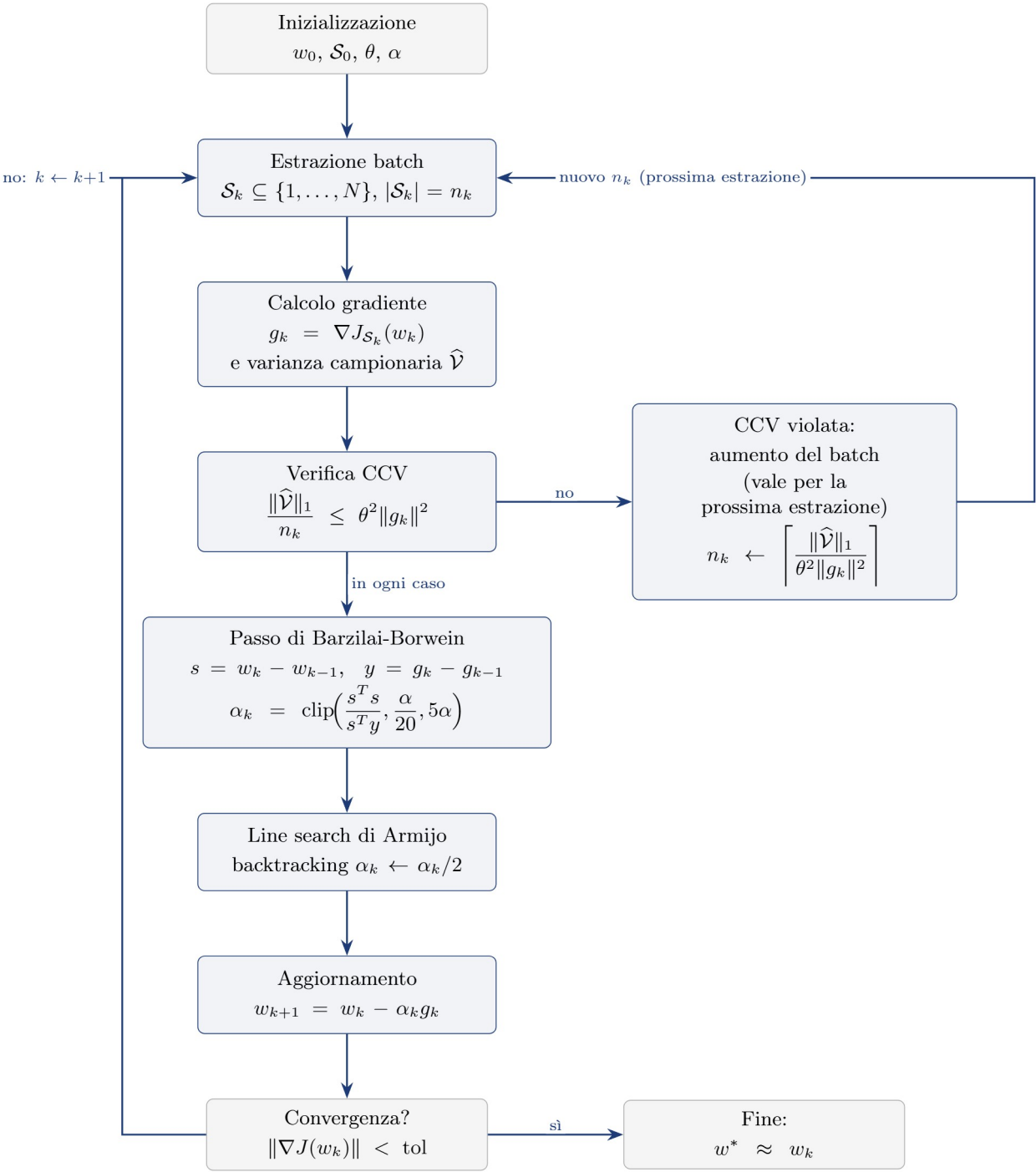
BB-CCV sostituisce il passo fisso del Dynamic GD con un passo adattivo che stima la curvatura di J lungo la direzione percorsa usando solo gradienti già calcolati: nessuna Hessiana

$w_{k+1} = w_k + \Delta w_k$ Tra due iterate consecutive w_{k+1} e w_k , lo sviluppo al primo ordine del gradiente $g = \nabla J$ attorno a w_k è
Posto $s_k := w_{k+1} - w_k$ (spostamento) e $y_k := g(w_{k+1}) - g(w_k)$ (variazione del gradiente), si ha la relazione secante $y_k \approx \nabla^2 J(w_k) s_k$
I metodi quasi-Newton approssimano l'Hessiana con B_{k+1} ($B_{k+1} s_k = y_k$): noi poniamo $B_{k+1} = \alpha_{k+1}^{-1} I$

Il sistema $s = \alpha y$ è sovradeterminato perché abbiamo m equazioni (una per componente di s e y) e un'incognita, lo scalare α :
lo risolviamo ai minimi quadrati, possiamo trovare $\min_{\alpha} \|\alpha y_k - s_k\|_2^2$ oppure $\min_{\alpha} \|\alpha^{-1} s_k - y_k\|_2^2$ ottenendo due formule diverse:

$$\alpha_k^{(1)} = \frac{s_k^T y_k}{y_k^T y_k}, \quad \alpha_k^{(2)} = \frac{s_k^T s_k}{s_k^T y_k}.$$

Schema BB-CCV



Esperimenti e visualizzazione interattiva

Funzioni di loss testate:

- Quadratica ben condizionata ($\kappa \approx 1.1$) $J(w) = (w_1 - 1)^2 + (w_2 + 2)^2 + 0.1w_1w_2$
- Quadratica mal condizionata ($\kappa \approx 20$) $J(w) = 20(w_1 - 1)^2 + (w_2 + 2)^2$
- Quadratica molto mal condizionata ($\kappa \approx 100$) $J(w) = 100(w_1 - 1)^2 + (w_2 + 2)^2$
- Quadratica con termine incrociato ($\kappa \approx 1.67$) $J(w) = (w_1 - 1)^2 + (w_2 + 2)^2 + 0.5(w_1 - 1)(w_2 + 2)$
- Funzione di Rosenbrock (non quadratica) $J(w) = \frac{1}{N} \sum_{i=1}^N \left[(w_1 - a_i)^2 + 100((w_2 - b_i) - (w_1 - a_i)^2)^2 \right]$

Applicazione Web: [visualizzazione.html](#)

Pyodide (Python nel browser) + Plotly

Algoritmi e loss personalizzabili

Traiettorie e metriche interattive

$N = 200$

$w_0 = (2, -3)$

seed = 42

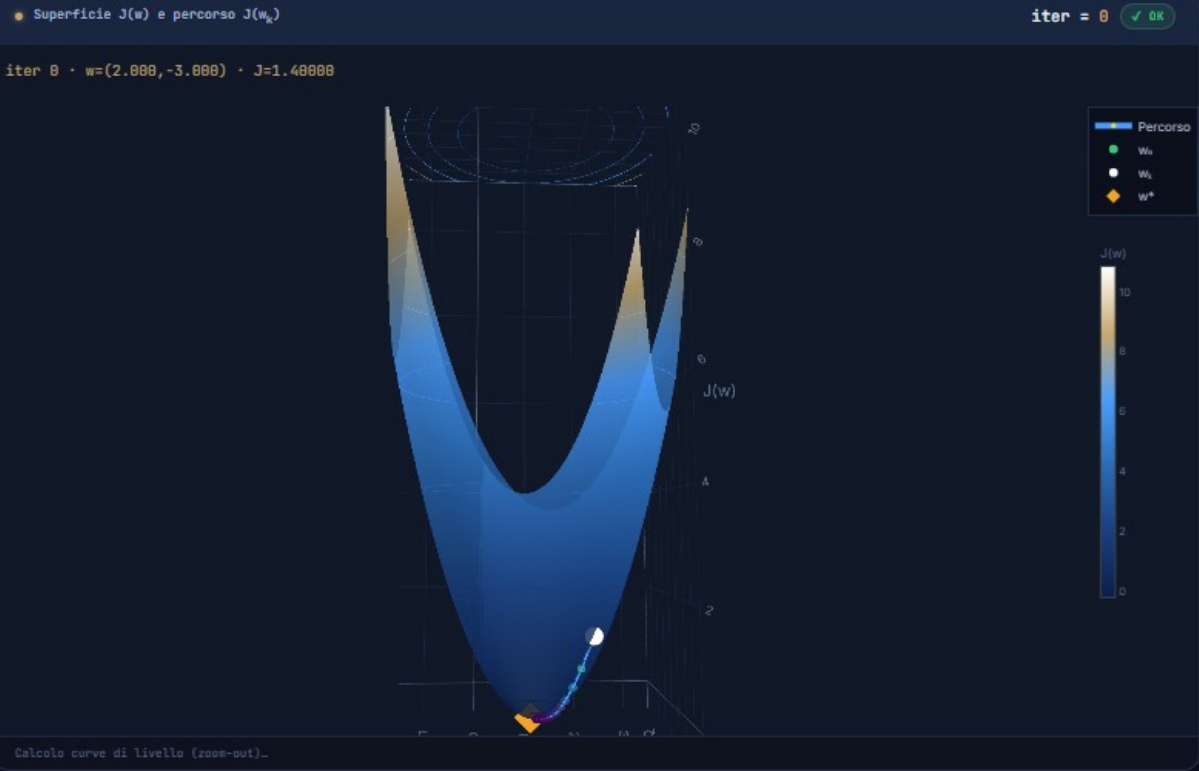
$\alpha = 0.1$

$\theta = 0.5$

$n_0 = 5$

max iter = 30

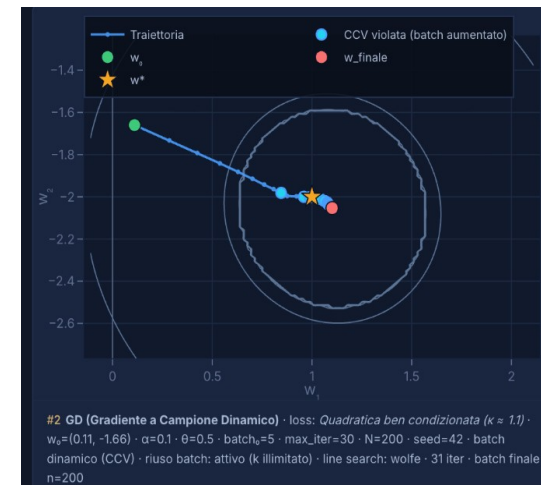
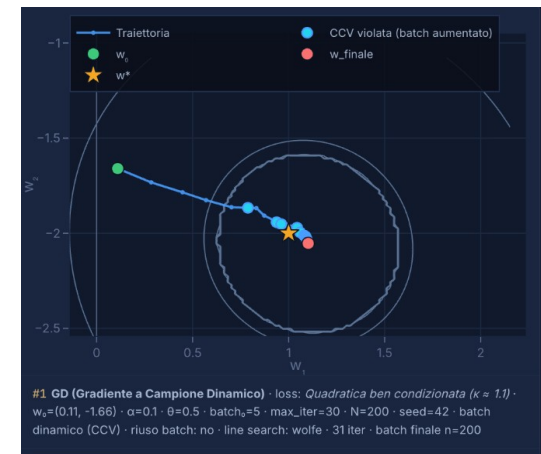
tolleranza = 10^{-6}



Riuso dello stesso minibatch

Tabella 6.1: Errore finale $e_{30} = \|w_{30} - w_*\|_2$ al termine delle 30 iterazioni (seed 42) per tutti i valori del numero massimo M di iterazioni consecutive sullo stesso mini-batch. *base*: ricampionamento a ogni iterazione. Simboli relativi a *base*: $\blacktriangle$ = migliora, $\blacktriangledown$ = peggiora, $=$ = invariato. Valori arrotondati a tre cifre significative.

Metodo	<i>base</i>	$M=\infty$	$M=10$	$M=5$	$M=3$	$M=2$	$M=1$
$\kappa \approx 1.1$ - Dynamic GD	1.1895×10^{-1}	$1.1739 \times 10^{-1} \blacktriangle$	$1.1739 \times 10^{-1} \blacktriangle$	$1.1920 \times 10^{-1} \blacktriangledown$	$1.1903 \times 10^{-1} \blacktriangledown$	$1.1777 \times 10^{-1} \blacktriangle$	$1.1895 \times 10^{-1} =$
$\kappa \approx 1.1$ - BB-CCV	1.1662×10^{-1}	$1.1662 \times 10^{-1} \blacktriangle$	$1.1662 \times 10^{-1} \blacktriangle$	$1.1662 \times 10^{-1} \blacktriangle$	$1.1662 \times 10^{-1} \blacktriangle$	$1.1662 \times 10^{-1} \blacktriangle$	$1.1662 \times 10^{-1} =$
$\kappa \approx 1.1$ - Newton-CG	4.0264×10^{-1}	$2.2230 \times 10^{-1} \blacktriangle$	$3.7373 \times 10^{-1} \blacktriangle$	$4.1553 \times 10^{-1} \blacktriangledown$	$3.2913 \times 10^{-1} \blacktriangle$	$2.6069 \times 10^{-1} \blacktriangle$	$3.0560 \times 10^{-1} \blacktriangle$
$\kappa \approx 1.1$ - Newton-CG L_1	1.0167×10^{-1}	$8.1750 \times 10^{-2} \blacktriangle$	$9.2407 \times 10^{-2} \blacktriangle$	$1.0010 \times 10^{-1} \blacktriangle$	$1.0624 \times 10^{-1} \blacktriangledown$	$9.4078 \times 10^{-2} \blacktriangle$	$1.0028 \times 10^{-1} \blacktriangle$
$\kappa \approx 20$ - Dynamic GD	4.9050×10^{-2}	$1.0103 \times 10^{-1} \blacktriangledown$	$7.6020 \times 10^{-2} \blacktriangledown$	$4.7211 \times 10^{-2} \blacktriangle$	$1.1746 \times 10^{-1} \blacktriangledown$	$4.2898 \times 10^{-2} \blacktriangle$	$4.9050 \times 10^{-2} =$
$\kappa \approx 20$ - BB-CCV	9.7195×10^{-5}	$9.5364 \times 10^{-10} \blacktriangle$	$9.5364 \times 10^{-10} \blacktriangle$	$2.6835 \times 10^{-3} \blacktriangledown$	$3.8422 \times 10^{-8} \blacktriangle$	$1.5436 \times 10^{-10} \blacktriangle$	$9.7195 \times 10^{-5} =$
$\kappa \approx 20$ - Newton-CG	2.3463×10^{-1}	$1.2807 \times 10^0 \blacktriangledown$	$9.4933 \times 10^{-1} \blacktriangledown$	$4.1051 \times 10^{-1} \blacktriangledown$	$3.3396 \times 10^{-1} \blacktriangledown$	$4.0617 \times 10^{-1} \blacktriangledown$	$1.6683 \times 10^{-1} \blacktriangle$
$\kappa \approx 20$ - Newton-CG L_1	4.8802×10^{-1}	$7.6221 \times 10^{-1} \blacktriangledown$	$5.5687 \times 10^{-1} \blacktriangledown$	$6.2190 \times 10^{-1} \blacktriangledown$	$4.5512 \times 10^{-1} \blacktriangledown$	$4.3392 \times 10^{-1} \blacktriangle$	$5.4573 \times 10^{-1} \blacktriangledown$
$\kappa \approx 100$ - Dynamic GD	4.6668×10^{-1}	$3.7060 \times 10^{-1} \blacktriangle$	$3.7060 \times 10^{-1} \blacktriangle$	$3.7060 \times 10^{-1} \blacktriangle$	$3.7060 \times 10^{-1} \blacktriangle$	$3.7060 \times 10^{-1} \blacktriangle$	$4.6668 \times 10^{-1} =$
$\kappa \approx 100$ - BB-CCV	1.0991×10^{-14}	$7.4538 \times 10^{-3} \blacktriangledown$	$7.4538 \times 10^{-3} \blacktriangledown$	$7.4538 \times 10^{-3} \blacktriangledown$	$7.4538 \times 10^{-3} \blacktriangledown$	$7.4538 \times 10^{-3} \blacktriangledown$	$1.0991 \times 10^{-14} =$
$\kappa \approx 100$ - Newton-CG	2.0969×10^{-1}	$1.2807 \times 10^0 \blacktriangledown$	$6.4441 \times 10^{-1} \blacktriangledown$	$3.3452 \times 10^{-1} \blacktriangledown$	$3.7338 \times 10^{-1} \blacktriangledown$	$4.4283 \times 10^{-1} \blacktriangledown$	$2.3041 \times 10^{-1} \blacktriangledown$
$\kappa \approx 100$ - Newton-CG L_1	9.6954×10^{-1}	$9.7065 \times 10^{-1} \blacktriangledown$	$8.6360 \times 10^{-1} \blacktriangledown$	$9.6997 \times 10^{-1} \blacktriangledown$	$8.7711 \times 10^{-1} \blacktriangledown$	$9.6944 \times 10^{-1} \blacktriangle$	$8.6562 \times 10^{-1} \blacktriangle$
incrociato ($\kappa \approx 1.67$) - Dynamic GD	9.4161×10^{-3}	$5.5315 \times 10^{-3} \blacktriangle$	$9.9566 \times 10^{-3} \blacktriangledown$	$5.9490 \times 10^{-3} \blacktriangle$	$7.5801 \times 10^{-3} \blacktriangle$	$8.7213 \times 10^{-3} \blacktriangle$	$9.4161 \times 10^{-3} =$
incrociato ($\kappa \approx 1.67$) - BB-CCV	6.1644×10^{-4}	$6.1689 \times 10^{-4} \blacktriangledown$	$6.1689 \times 10^{-4} \blacktriangledown$	$6.1689 \times 10^{-4} \blacktriangledown$	$6.1610 \times 10^{-4} \blacktriangle$	$6.1685 \times 10^{-4} \blacktriangledown$	$6.1644 \times 10^{-4} =$
incrociato ($\kappa \approx 1.67$) - Newton-CG	3.5167×10^{-1}	$4.1026 \times 10^{-2} \blacktriangle$	$1.0511 \times 10^{-1} \blacktriangle$	$3.3540 \times 10^{-1} \blacktriangle$	$2.3364 \times 10^{-1} \blacktriangle$	$2.1820 \times 10^{-1} \blacktriangle$	$2.6381 \times 10^{-1} \blacktriangle$
incrociato ($\kappa \approx 1.67$) - Newton-CG L_1	2.0877×10^{-2}	$3.8938 \times 10^{-2} \blacktriangledown$	$3.9410 \times 10^{-2} \blacktriangledown$	$2.9304 \times 10^{-2} \blacktriangledown$	$2.3645 \times 10^{-2} \blacktriangledown$	$3.8543 \times 10^{-2} \blacktriangledown$	$2.5154 \times 10^{-2} \blacktriangledown$
Funzione di Rosenbrock ($c=100$) - Dynamic GD	6.1354×10^{-4}	$2.5435 \times 10^0 \blacktriangledown$	$2.5435 \times 10^0 \blacktriangledown$	$2.5435 \times 10^0 \blacktriangledown$	$2.0803 \times 10^0 \blacktriangledown$	$1.4213 \times 10^0 \blacktriangledown$	$6.1354 \times 10^{-4} =$
Funzione di Rosenbrock ($c=100$) - BB-CCV	1.0645×10^{-1}	$1.6648 \times 10^{-1} \blacktriangledown$	$1.6648 \times 10^{-1} \blacktriangledown$	$1.6648 \times 10^{-1} \blacktriangledown$	$1.6648 \times 10^{-1} \blacktriangledown$	$3.6984 \times 10^{-1} \blacktriangledown$	$1.0645 \times 10^{-1} =$
Funzione di Rosenbrock ($c=100$) - Newton-CG	2.0625×10^{-1}	$1.0605 \times 10^0 \blacktriangledown$	$1.1367 \times 10^0 \blacktriangledown$	$1.2475 \times 10^0 \blacktriangledown$	$1.1893 \times 10^0 \blacktriangledown$	$1.2292 \times 10^0 \blacktriangledown$	$9.3953 \times 10^{-1} \blacktriangledown$
Funzione di Rosenbrock ($c=100$) - Newton-CG L_1	1.2525×10^{-1}	$3.8199 \times 10^{-1} \blacktriangledown$	$2.4883 \times 10^{-1} \blacktriangledown$	$3.0766 \times 10^{-1} \blacktriangledown$	$3.1242 \times 10^{-1} \blacktriangledown$	$2.7111 \times 10^{-1} \blacktriangledown$	$2.0207 \times 10^{-1} \blacktriangledown$



Robustezza del riuso

Tabella 6.2: Robustezza del riuso del mini-batch su 5 seed indipendenti ($\{42, 7, 123, 2024, 999\}$): per ogni coppia problema-algoritmo e per ciascuna politica di riuso ($M=\infty$ e $M=10$), numero di seed su 5 in cui e_{30} rispetto alla *base* è minore (*Migl.*), maggiore (*Pegg.*) o identico (*Uguale*).

Problema	Algoritmo	$M=\infty$			$M=10$		
		<i>Migl.</i>	<i>Pegg.</i>	<i>Uguale</i>	<i>Migl.</i>	<i>Pegg.</i>	<i>Uguale</i>
$\kappa \approx 1.1$	Dynamic GD	5	0	0	5	0	0
	BB-CCV	5	0	0	5	0	0
	Newton-CG	4	1	0	3	2	0
	Newton-CG L_1	4	1	0	5	0	0
$\kappa \approx 20$	Dynamic GD	2	3	0	1	4	0
	BB-CCV	3	2	0	3	2	0
	Newton-CG	0	5	0	0	5	0
	Newton-CG L_1	1	4	0	2	3	0
$\kappa \approx 100$	Dynamic GD	3	2	0	3	2	0
	BB-CCV	1	3	1	1	3	1
	Newton-CG	0	5	0	0	5	0
	Newton-CG L_1	3	2	0	4	1	0
incrociato ($\kappa \approx 1.67$)	Dynamic GD	4	1	0	3	2	0
	BB-CCV	2	3	0	2	3	0
	Newton-CG	4	1	0	3	2	0
	Newton-CG L_1	2	3	0	1	4	0
Funzione di Rosenbrock ($c=100$)	Dynamic GD	2	3	0	2	3	0
	BB-CCV	1	4	0	1	4	0
	Newton-CG	1	4	0	2	3	0
	Newton-CG L_1	1	4	0	2	3	0

Stop adattivo con validation set

Il riuso illimitato ($M = \infty$) può far collassare i metodi di Newton: l'Hessiana "invecchia" e la direzione di Newton si allontana dalla discesa, fino a bloccarsi (errore ≈ 1.28 su $\kappa \approx 20$, Tabella 6.1). Non esiste un valore di M ottimale per tutti i problemi e algoritmi.

- Si riserva una frazione p del dataset come validation set.
- Il mini-batch è campionato solo dal training set.
- Ogni f iterazioni, si valuta la loss sul validation set: $J_{\text{val}}(w_{k+1})$.
- Se per P valutazioni consecutive non c'è miglioramento, il batch viene ricampionato.
- Criterio di miglioramento: $J_{\text{val}}(w_{k+1}) \leq J_{\text{val}}^{\text{best}}(1 - \tau) - \epsilon_{\text{abs}}$

Tabella 6.4: Robustezza su 5 seed indipendenti: $\bar{e}_{30}$ = media di e_{30} su 20 caselle $\times$ 5 seed; *vitt.* = caselle (su 20) in cui la media su 5 seed è minore di quella di *base*; le ultime cinque colonne riportano la media su 5 seed per problema.

Configurazione	$\bar{e}_{30}$	<i>vitt.</i>	$\kappa \approx 1.1$	$\kappa \approx 20$	$\kappa \approx 100$	incr. ($\kappa \approx 1.67$)	Funzione di Rosenbrock ($c=100$)
<i>base</i>	2.1608×10^{-1}	0/20	1.8850×10^{-1}	1.9229×10^{-1}	4.0422×10^{-1}	8.4365×10^{-2}	2.1102×10^{-1}
$M=\infty$	3.8354×10^{-1}	5/20	2.4978×10^{-1}	4.5080×10^{-1}	7.1556×10^{-1}	1.6970×10^{-1}	3.3186×10^{-1}
def.	2.6706×10^{-1}	7/20	1.3485×10^{-1}	3.1368×10^{-1}	5.7133×10^{-1}	3.8100×10^{-2}	2.7736×10^{-1}
$P=1, p=0.1, \text{dyn}$	2.0375×10^{-1}	12/20	1.4045×10^{-1}	2.1065×10^{-1}	4.5437×10^{-1}	3.9223×10^{-2}	1.7407×10^{-1}
$P=3, p=0.1, \text{dyn}$	2.2931×10^{-1}	11/20	1.3955×10^{-1}	2.4684×10^{-1}	5.2925×10^{-1}	4.3722×10^{-2}	1.8717×10^{-1}

Riuso per discesa della loss sul batch

Lo stop adattivo con validation set richiede un calcolo per ogni dato nell'insieme
Nessun dato riservato: il mini-batch è campionato dall'intero dataset.
La decisione di ricampionare si basa sulla loss sul batch corrente $J_{\text{batch}}(w_k)$.
La line search garantisce che J_{batch} scenda sempre: il criterio scatta quando la riduzione è lenta (vicino al minimo campionario).
Ogni f iterazioni, si confronta la loss corrente con quella dell'iterazione precedente sullo stesso batch.

P valutazioni senza miglioramento $\rightarrow$ ricampiona
 $J_{\text{batch}}(w_{k-1}) - J_{\text{batch}}(w_k) \geq \tau |J_{\text{batch}}(w_{k-1})| + \epsilon_{\text{abs}}$

Tabella 6.7: Confronto finale su 5 seed indipendenti (20 combinazioni problema×algoritmo) tra *base*, $M=\infty$, lo stop adattivo con validation set calibrato ($P=1$, $p=10\%$, split dinamico) e il riuso per discesa calibrato ($\tau=10^{-3}$, $P=1$, $f=1$). $\bar{e}_{30}$ = media su 20 caselle $\times$ 5 seed; *vitt.* = caselle (su 20) in cui la media su 5 seed è minore del proprio riferimento.

Strategia	$\bar{e}_{30}$	<i>vitt.</i>	$\kappa \approx 1.1$	$\kappa \approx 20$	$\kappa \approx 100$	incr. ($\kappa \approx 1.67$)	Funzione di Rosenbrock ($c=100$)
<i>base</i>	2.0909×10^{-1}	0/20	1.7930×10^{-1}	1.7595×10^{-1}	4.1162×10^{-1}	8.1419×10^{-2}	1.9719×10^{-1}
$M=\infty$	3.7299×10^{-1}	8/20	1.9159×10^{-1}	5.0123×10^{-1}	6.6432×10^{-1}	9.8773×10^{-2}	4.0901×10^{-1}
$P=1, p=0.1, \text{dyn}$	2.0375×10^{-1}	12/20	1.4045×10^{-1}	2.1065×10^{-1}	4.5437×10^{-1}	3.9223×10^{-2}	1.7407×10^{-1}
$\tau=10^{-3}, P=1, f=1$	2.3817×10^{-1}	11/20	1.1855×10^{-1}	2.3416×10^{-1}	4.2013×10^{-1}	3.4413×10^{-2}	3.8362×10^{-1}

Conclusioni

Il batch dinamico è adattivo e competitivo con lo stato dell'arte.

- BB-CCV raggiunge la precisione macchina su $\kappa \approx 100$: $e_{30} = 1.10 \times 10^{-14}$
- Dynamic GD è il più stabile su Rosenbrock: $e_{30} = 6.14 \times 10^{-4}$
- Newton-CG e Newton-CG L1 soffrono su Rosenbrock (Hessiana sottocampionata poco rappresentativa)

Il riuso del batch aiuta sui ben condizionati, ma va controllato sui mal condizionati e sui metodi del secondo ordine.

- Su $\kappa \approx 1.1$: riuso migliora su 5/5 seed
- Su $\kappa \approx 20$: Newton-CG con $M = \infty$ collassa ($e_{30} \approx 1.28$)
- BB-CCV su $\kappa \approx 20$: riuso porta da 10^{-5} a 10^{-10}

Lo stop adattivo migliora i metodi di Newton.

- L'app web permette di sperimentare i metodi in tempo reale.

Lavoro futuro

1. Rilassamento dell'ipotesi di convessità forte

Sostituire la convessità forte con la condizione di Polyak–Łojasiewicz (PL):

La convessità forte è un caso particolare della PL

In questo modo l'analisi teorica coprirebbe un insieme più ampio di funzioni

2. Rapporto di sottocampionamento dell'Hessiana dinamico

Anziché tenere R fisso, si potrebbe aumentarlo quando la varianza dell'Hessiana cresce

3. Implementazione e confronto sistematico con GD, SGD, SVRG e SAGA

Implementare i metodi GD, SGD, SVRG e SAGA nella stessa piattaforma (PyTorch/NumPy) e confrontarli con il campionamento dinamico in termini di costo computazionale totale (numero di gradienti valutati o tempo di esecuzione), non di iterazioni.

Grazie per l'attenzione

Domande?